

SpringBoot 3.x + SpringFramework 6.x 의 추가된 ProblemDetail 클래스

- : Http API 에서 나올 수 있는 error를 표준화된 방식으로 제공
- : JSON과 XML 형식으로, "어떠한 문제가 발생하였는지"를 명시하기 위한 스펙
- : IETF 의 RFC 7807 (=Problem Details for HTTP APIs)

IETF의 **RFC**(Request for Comments)는 인터넷과 관련된 기술 및 프로토콜, 정책에 대한 공식 문서이다. IETF(Internet Engineering Task Force)는 인터넷 표준과 관련된 기술을 개발하고 발전시키는 조직으로, RFC는 이들이 제안하거나 승인한 표준, 권장 사항, 정보 문서 등을 기록한 문서이다

👉 나온(사용하는) 이유?

Spring Framework 를 사용하여 웹을 구현하면 기본적으로 제공되는 에러처리에 대한 에러 응답 결과는 다음과 같은 형태이다.

```
{
  "timestamp":1417379464584,
  "status":400,
  "error":"Bad Request",
  "exception":"org.springframework.web.bind.MethodArgumentNotValidException",
  "message":"Validation failed for argument at index 0 in method: public org.springframework
  "path":"/testUser"
}
```

그대로 사용하기에는 외부에 공개되지 않았으면 하는 정보가 있거나 개발 확인용 내용이 많기 때문에 보안적인 이슈가 있을 수 있다.

물론,

Spring Boot 를 사용하는 경우 다음과 같은 ErrorProperties 설정을 통해 노출할 내용을 적절히 조절할 수도 있다.

```
server.error.include-exception=false  
server.error.include-stacktrace=never  
server.error.include-binding-errors=never
```

해당 설정을 통해 개발 시엔 개발 관련 내용을 확인하면서 서비스 중일 땐 해당 내용을 숨기는 것이 가능하다.

하지만,

사용자에게 전달하기 위해 좀 더 상세한 에러 메시지를 처리하려고 하는 경우 기본으로 제공해 주는 이 에러 처리를 사용하기 불편하여 에러 메시지 객체를 따로 정의하고 exception handler 를 통해 해당 에러를 응답으로 내려보내주는 식으로 에러를 관리하는 경우가 많았다.

👉 RFC 7807 - Problem Details for HTTP APIs

RFC 7807 은 API 에러 응답에 대한 규약을 정의한 문서이고 현재 Standard Track 으로 표준화되었다고 보면 된다.

<https://www.rfc-editor.org/rfc/rfc7807.html>

정리를 하면,

에러 발생 시 안내할 응답 내용을 문서 세부 정보 개체 ("type", "title", "status", "detail", "instance")와 확장 멤버로 구성하자는 것이다.

복잡한 구현이 있는 게 아니고 '이렇게 합시다'라는 약속을 정의한 것이다.

Spring Framework 6.0.x 의 spring-web 모듈에 다음과 같이 RFC 7807 을 따르는 ProblemDetail class 가 추가되었다.

<https://github.com/spring-projects/spring-framework/blob/main/spring-web/src/main/java/org/springframework/http/ProblemDetail.java>

RFC 7807 Problem Detail for API 문서 이름의 응답 객체 명세 이름을 그대로 사용하여 정의하였다.

☞ ProblemDetail에서 제공하는 응답형태의 항목

- **type (string, URI)**: 이 문제에 대한 자세한 원인을 알려주는 문서(document)의 위치, 명시되지 않은 경우 "about:blank"로 추정
- **title (string)**: 문제 원인을 간결하게 표현한 제목
- **status (integer, optional)** : HttpStatus Code
- **detail (string, optional)** : 자세하게 문제의 원인을 작성
- **instance (string, URI, optional)** : 문제를 일으키는 요청 URL

API

<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org.springframework/http/ProblemDetail.html>

@RestControllerAdvice

<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org.springframework/web/bind/annotation/RestControllerAdvice.html>

Package org.springframework.web.bind.annotation

Annotation Interface RestControllerAdvice

@Target (TYPE)

@Retention (RUNTIME)

@Documented

@ControllerAdvice

@ResponseBody

public @interface RestControllerAdvice

A convenience annotation that is itself annotated with @ControllerAdvice and @ResponseBody.

Types that carry this annotation are treated as controller advice where @ExceptionHandler methods assume @ResponseBody semantics by default.

Spring 은 **전역적으로 예외를 처리할 수 있는 @ControllerAdvice 와**

@RestControllerAdvice 어노테이션을 각각 Spring3.2, Spring4.3 부터 제공하고 있다. 두 개의 차이는 @Controller 와 RestController 와 같은데, @RestControllerAdvice 는 @ControllerAdvice 와 달리 @ResponseBody 가 붙어 있어 응답을 Json 으로 내려준다는 점에서 다르다.

주요기능에 따른 예외처리

- 1) **회원가입** - MemberAuthenticationException(message, title)
- 2) **전체검색** - BoardSearchNotException(message, title, status)
- 3) **상세보기** - BoardSearchNotException(message, title, status)

- 4) 글등록- DMLException(message, title, status)
- 5) 삭제하기 - DMLException(message, title, status)
- 6) 수정하기 -DMLException(message, title, status)

HttpStatus API

<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/http/HttpStatus.html>

소스코드

```

6      @Getter
7      public class MemberAuthenticationException extends RuntimeException{
8          private String message;
9          private HttpStatus httpStatus;
10         private String title;
11         1 usage
12         public MemberAuthenticationException(String message, String title){
13             this(message , title, HttpStatus.UNAUTHORIZED);//
14         }
15         1 usage
16         public MemberAuthenticationException(String message, String title, HttpStatus httpStatus){
17             this.message=message;
18             this.title=title;
19             this.httpStatus=httpStatus;
20         }
21     }

```

```

@Getter
public class BoardSearchNotException extends RuntimeException{
    private String message;
    private HttpStatus httpStatus;
    private String title;
    2 usages
    public BoardSearchNotException(String message, String title){
        this(message, title, HttpStatus.EXPECTATION_FAILED);//417
    }
    1 usage
    public BoardSearchNotException(String message, String title, HttpStatus httpStatus){
        this.message=message;
        this.title=title;
        this.httpStatus=httpStatus;
    }
}

```

```

3 @RestControllerAdvice
4 public class DefaultExceptionHandler {
5     @ExceptionHandler({MemberAuthenticationException.class})
6     public ProblemDetail signInExceptionHandler(MemberAuthenticationException e){
7         ProblemDetail problemDetail = ProblemDetail.forStatus(e.getHttpStatus().value());
8
9         problemDetail.setTitle(e.getTitle());
10        problemDetail.setDetail(e.getMessage());
11        problemDetail.setProperty("timestamp", LocalDateTime.now());
12
13        return problemDetail;
14    }
15 }
16 }

```

```

@RestControllerAdvice
public class DefaultExceptionHandler {
    @ExceptionHandler({BoardSearchNotException.class})
    public ProblemDetail boardSearchExceptionHandler(BoardSearchNotException e){
        ProblemDetail problemDetail = ProblemDetail.forStatus(e.getHttpStatus().value());
        problemDetail.setTitle(e.getTitle());
        problemDetail.setDetail(e.getMessage());
        problemDetail.setProperty("timestamp", LocalDateTime.now());
        return problemDetail;
    }

    @ExceptionHandler(Exception.class)
    public ProblemDetail exceptionHandle(Exception e){
        ProblemDetail problemDetail = ProblemDetail.forStatus(600);
        problemDetail.setTitle("DB error");
        problemDetail.setDetail("예외가 발생하였습니다.");
        problemDetail.setProperty("timestamp", LocalDateTime.now());
        return problemDetail;
    }
}

```

Service

2 usages

```
@Transactional(readOnly = true)
public List<Board> findAllBoard() throws BoardSearchNotException {
    List<Board> list = boardRepository.findAll();
    if(list == null || list.isEmpty())
        throw new BoardSearchNotException("전체게시물이 없습니다.", "Not Found Board All");

    return list;
}
```

1 usage

```
@Transactional
public Board updateBoard(Long id, Board board) throws DMLEException {
    Board boardEntity = boardRepository.findById(id)
        .orElseThrow(() -> new DMLEException("글번호로 오류로 수정되지 않았습니다.", "Not Update"));

    boardEntity.setTitle(board.getTitle());
    boardEntity.setAuthor(board.getAuthor());
    return boardEntity;
}
```